

考点 26:顺序查找法&折半查找法

	• 小题9		• 小题11	• 应用题42		• 小题7	• 小题9	• 小题8
2009	2010	2011	2012	2013	2014	2015	2016	2017
						• 小题8	• 小题5	• 小题7 • 算法题41
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频： 小题×8、应用题×1、算法题——很多都需要使用顺序查找

顺序查找法&折半查找法

【考点笔记】顺序查找法

逐个比较查找表中的元素，直到找到目标元素或达到查找表的尽头为止，时间复杂度为 $O(n)$ 。

查找成功时，在等概率的情况下，平均查找长度为

$$ASL_{\text{成功}} = \sum_{i=1}^n P_i(n-i+1) = \frac{n+1}{2}$$

查找不成功时，平均查找长度为 $ASL_{\text{不成功}} = n+1$ 。 注：如果不使用“哨兵”的实现方法，则 $ASL_{\text{失败}} = n$

注意：有序线性表的顺序查找在查找失败时的平均查找长度是不同的。



数据从下标1开始存

16	33	10	13	29	16	19	32	7	43	41	37			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

TableLen=11



顺序查找法的优化方法

方法一：表中元素有序，可减少查找失败时的平均查找长度

查找表中元素有序存放（递增/递减）

查找目标：

21

7

13

19

29

37

43

方法二：按被查概率降序排列，可减少查找成功时的平均查找长度

被查概率大的放在靠前位置

被查概率

7: 15%

13: 5%

19: 10%

29: 40%

37: 28%

43: 2%

29

37

7

19

13

43

$$ASL_{成功} = 1 \times 0.4 + 2 \times 0.28 + 3 \times 0.15 + 4 \times 0.1 + 5 \times 0.05 + 6 \times 0.02 = 2.18$$

顺序查找法&折半查找法

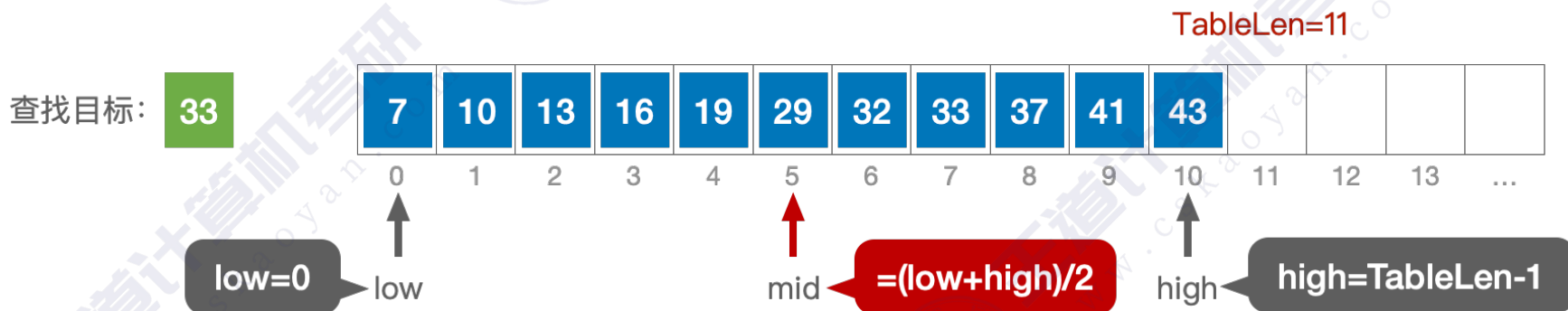
【考点笔记】折半查找法

又称二分查找法，它仅适用于有序的顺序表。折半查找的思想：

- ① 若顺序表为空，查找失败；否则，执行②。
- ② 将 key 与顺序表中间位置元素的关键字比较，若相等，查找成功；否则执行③。
- ③ 若 key 小于（大于）顺序表中间位置元素的关键字，则以中间元素的前半部分（后半部分）作为新的查找表，执行①。

与完全二叉树相同

描述折半查找过程的二叉树称为判定树，判定树的树高 $h = \lceil \log_2(n+1) \rceil$ 。表的中间结点是一棵树的根，左子表相当于左子树，右子表相当于右子树。



33 > mid, 只可能在右边区域

核心思想：对有序表进行中分！！

折半查找的实现

折半查找的代码一定要会写

```
typedef struct{
    ElemType *elem;
    int TableLen;
}SSTable;
```

//查找表的数据结构（顺序表）
//动态数组基址
//表的长度

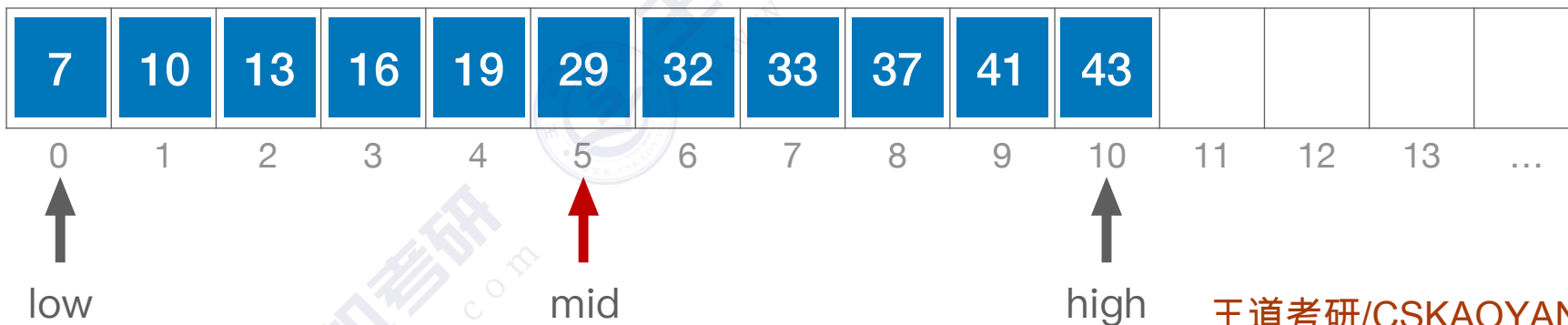
//折半查找

```
int Binary_Search(SSTable L, ElemType key){
    int low=0, high=L.TableLen-1, mid;
    while(low<=high){
        mid=(low+high)/2; //取中间位置
        if(L.elem[mid]==key) //查找成功则返回所在位置
            return mid;
        else if(L.elem[mid]>key) //从前半部分继续查找
            high=mid-1;
        else //从后半部分继续查找
            low=mid+1;
    }
    return -1; //查找失败，返回-1
}
```

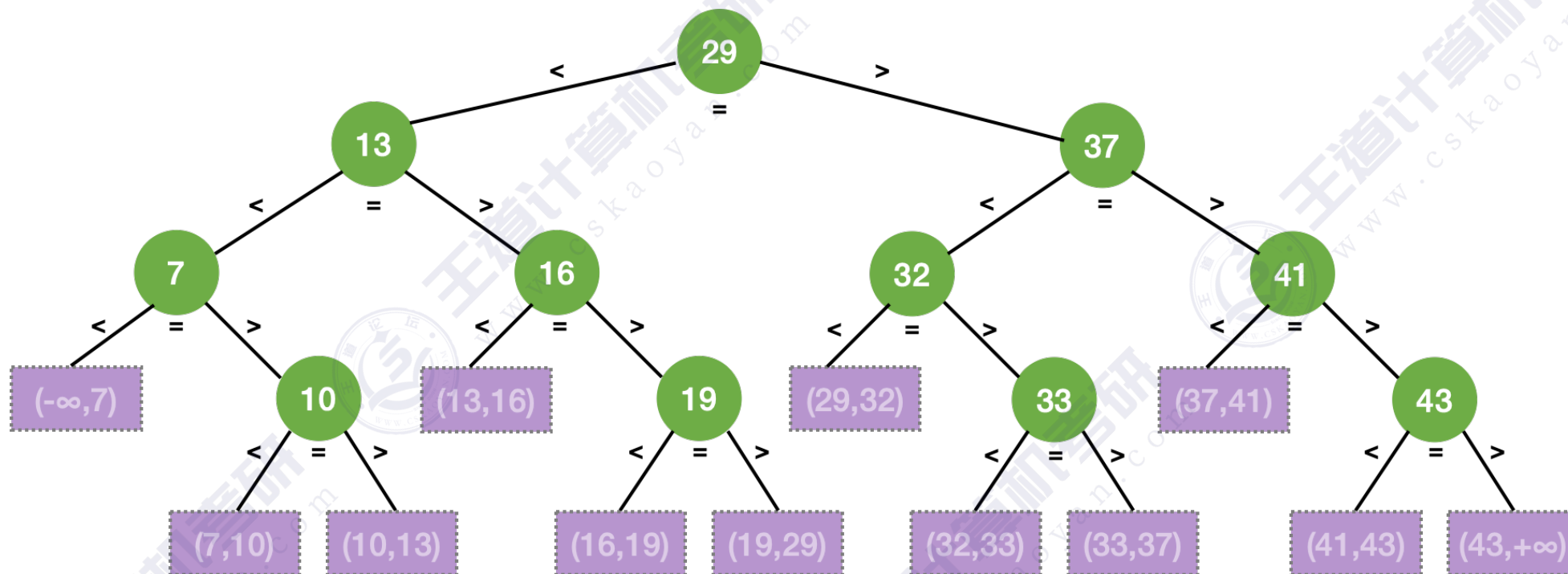
折半查找，又称“二分查找”，
仅适用于**有序**的**顺序表**。

顺序表拥有随机访问
的特性，链表没有

查找目标：33



折半查找的判定树



判定树结点关键字：左<中<右，满足二叉排序树的定义

失败结点：n+1个（等于成功结点的空链域数量）

树高 $h = \lceil \log_2(n + 1) \rceil$

折半查找时间复杂度= $O(\log_2 n)$

注：该树高不含失败结点

顺序查找法&折半查找法

9. 在有 n ($n > 1000$) 个元素的升序数组 A 中查找关键字 x 。查找算法的伪代码如下所示。

```
k=0;
while(k<n 且 A[k]<x) k=k+3;
if(k<n 且 A[k]==x) 查找成功;
else if(k-1<n 且 A[k-1]==x) 查找成功;
    else if(k-2<n 且 A[k-2]==x) 查找成功;
        else 查找失败;
```

本算法与折半查找算法相比，有可能具有更少比较次数的情形是_____。

- A. 当 x 不在数组中
- B. 当 x 接近数组开头处
- C. 当 x 接近数组结尾处
- D. 当 x 位于数组中间位置

7. 下列选项中，不能构成折半查找中关键字比较序列的是_____。

- A. 500,200,450,180
- B. 500,450,200,180
- C. 180,500,200,450
- D. 180,200,500,450

2016真题

2015真题

考点 27: 二叉排序树

		• 小题7		• 小题3 • 小题6		• 小题4		
2009	2010	2011	2012	2013	2014	2015	2016	2017
• 小题6		• 小题5		• 算法题 41		• 小题7		
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: 小题×7、算法题×1

二叉排序树

【考点笔记】二叉排序树的定义与特点

一棵二叉树或者是空二叉树，或者是具有如下性质的二叉树：

- ① 左子树上所有结点的关键字（若存在）均小于根结点的关键字。
- ② 右子树上所有结点的关键字（若存在）均大于根结点的关键字。
- ③ 左子树和右子树又各是一棵二叉排序树。

二叉排序树的特点：

中序遍历二叉排序树，可以得到一个递增的有序序列。

在二叉排序树中删除后又插入同一结点，得到的二叉排序树与原来的不一定相同，但中序序列一定相同，因为在二叉排序树中插入的新结点均作为叶子结点。

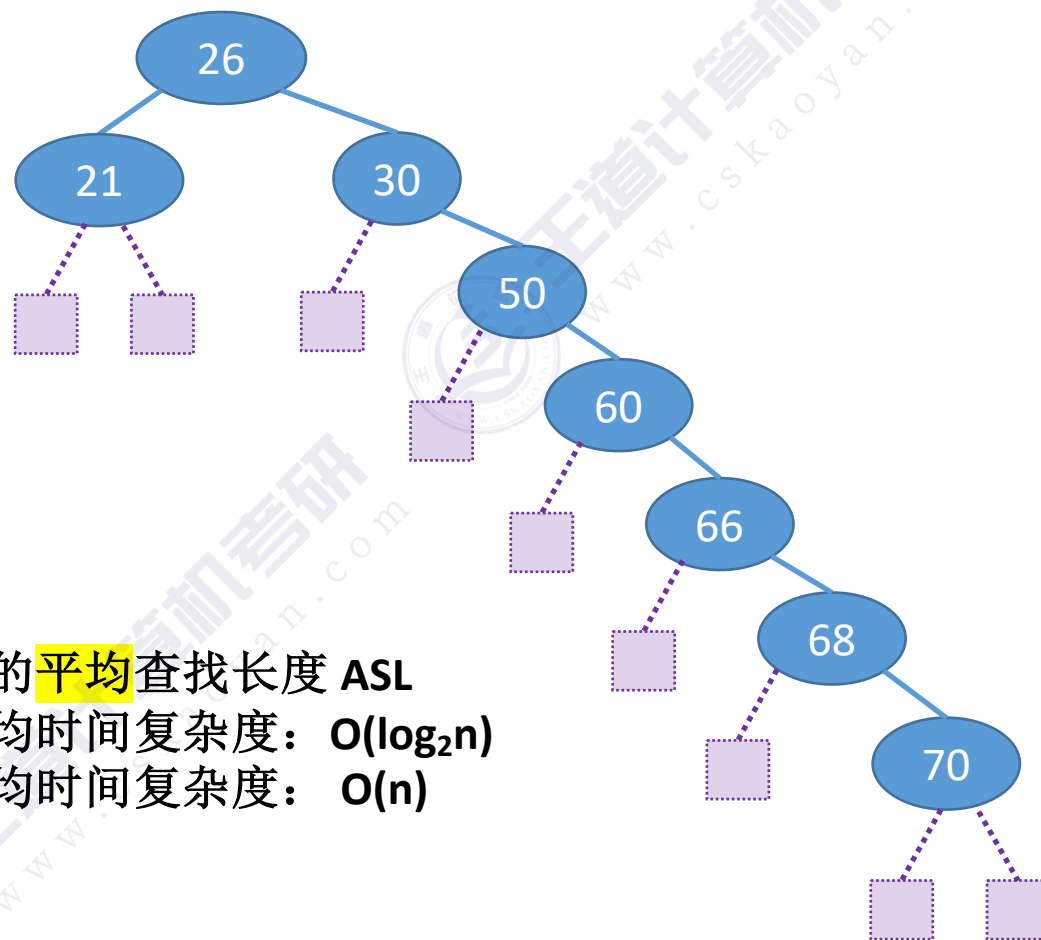
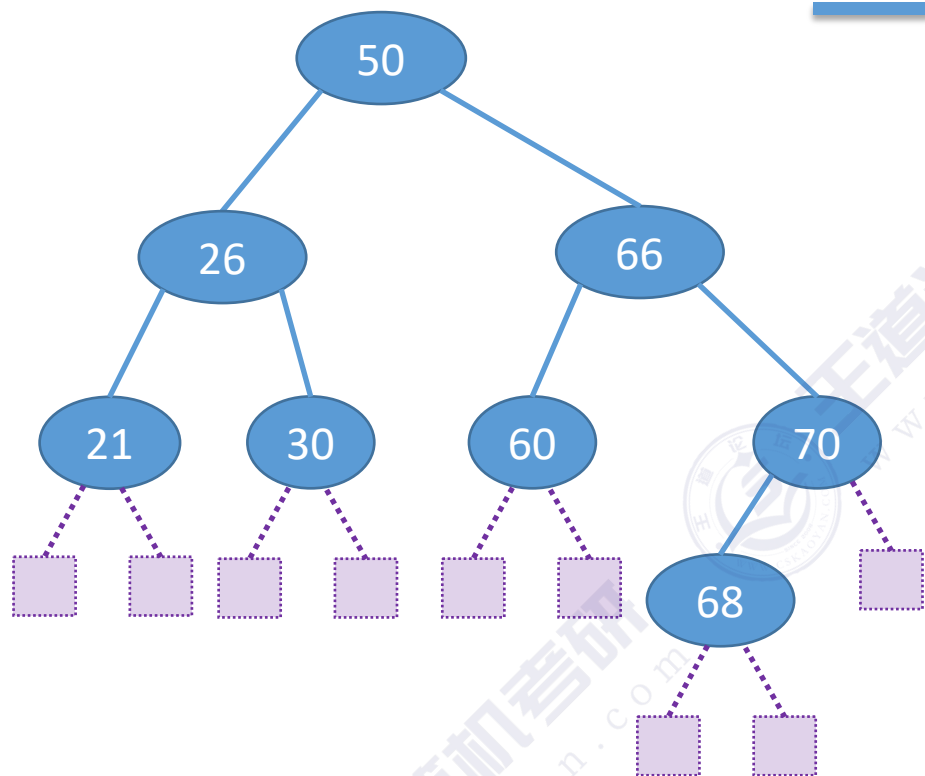
删除再插入叶子结点，则相同

左子树结点值 < 根结点值 < 右子树结点值



进行中序遍历，可以得到一个递增的有序序列

二叉排序树



回忆：查找失败的平均查找长度 ASL
最好情况下，平均时间复杂度： $O(\log_2 n)$
最坏情况下，平均时间复杂度： $O(n)$

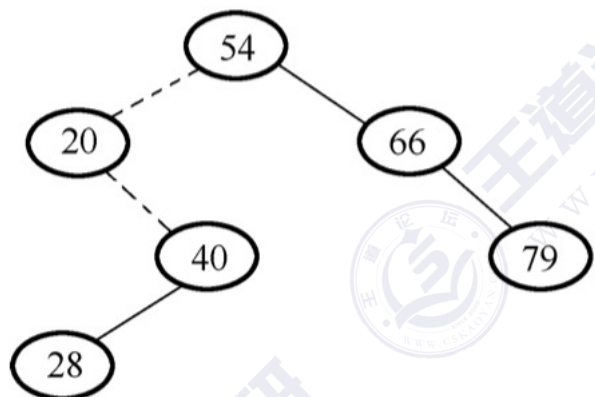
【考点笔记】二叉排序树的查找

二叉排序树的查找过程（若二叉排序树非空）：

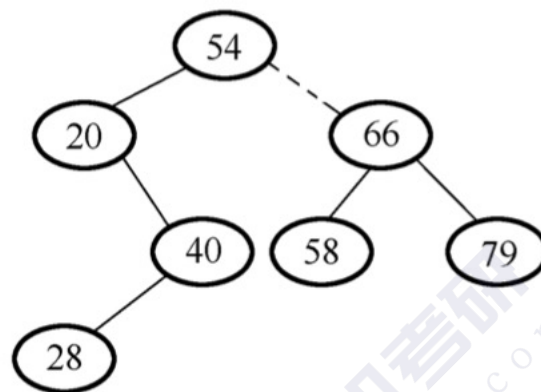
- ① 将给定值与根结点的关键字比较，若相等，则查找成功。
- ② 当根结点的关键字大于给定关键字值时，在根结点的左子树中查找。
- ③ 否则在根结点的右子树中查找。

二叉排序树

【考点笔记】二叉排序树的插入



(a) 插入28



(b) 插入58

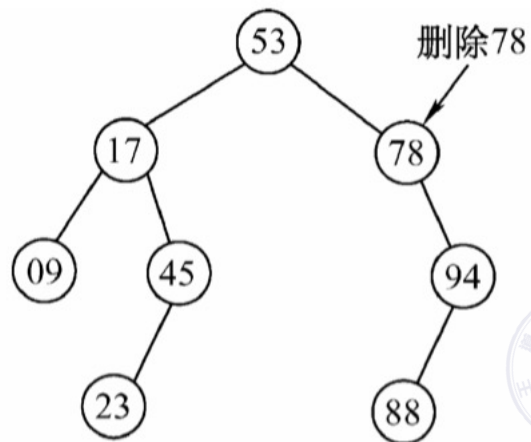
二叉排序树的插入过程（与查找过程类似）：

- ① 若原二叉排序树为空，则直接插入结点。
- ② 若关键字 k 小于根结点关键字，则插入到左子树中。
- ③ 若关键字 k 大于根结点关键字，则插入到右子树中。

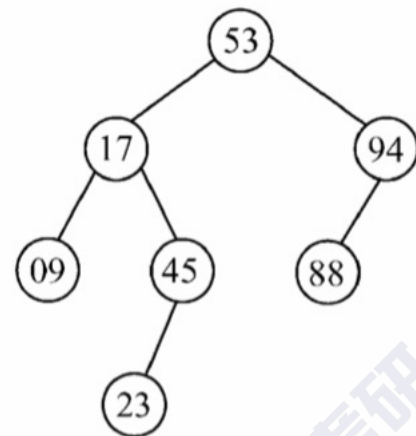
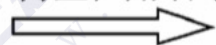
由此可见，插入的新结点一定是某个叶结点。

二叉排序树

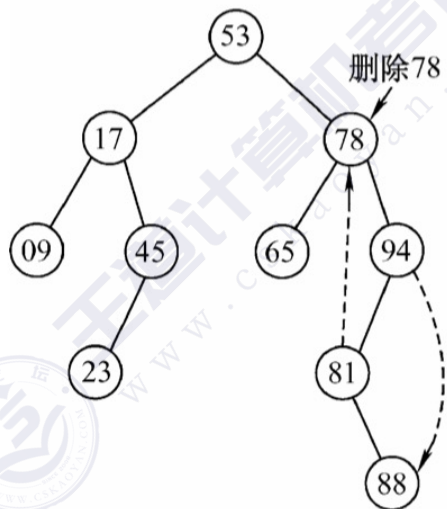
【考点笔记】二叉排序树的删除



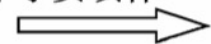
左子树空，用右子女填补



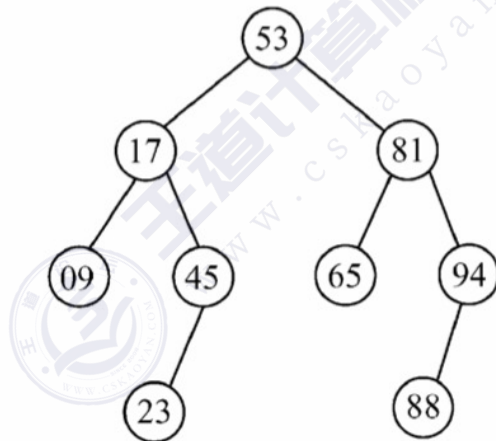
(a)



左、右子树均不空，
在右子树上找中序第
一个子女填补



转换为删除81



(b)

① 如果被删除结点 x 是叶结点，则直接删除。

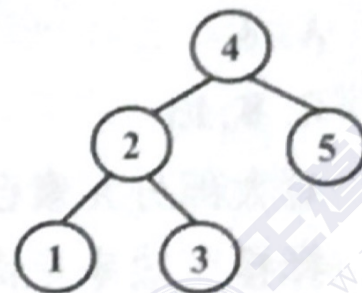
② 若结点 x 只有一棵左子树或右子树，则让 x 的子树成为 x 父结点的子树，替代 x 的位置。

③ 若结点 x 有左、右两棵子树，则令 x 的中序后继（或前驱）替代 x ，然后从二叉排序树中删去这个直接后继（或直接前驱），这样就转换成了第一或第二种情况。

二叉排序树

警惕：大题的考察，给你一棵排序树，算ASL、插入某节点、删除某节点的状态。

下列给定的关键字输入序列中，不能生成如下二叉排序树的是



2020真题

A. 4, 5, 2, 1, 3

B. 4, 5, 1, 2, 3

C. 4, 2, 5, 3, 1

D. 4, 2, 1, 3, 5

6. 在任意一棵非空二叉排序树 T_1 中，删除某结点 v 之后形成二叉排序树 T_2 ，再将 v 插入 T_2 形成二叉排序树 T_3 。下列关于 T_1 与 T_3 的叙述中，正确的是_____。

I. 若 v 是 T_1 的叶结点，则 T_1 与 T_3 不同

II. 若 v 是 T_1 的叶结点，则 T_1 与 T_3 相同

III. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 不同

IV. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 相同

A. 仅 I、III

B. 仅 I、IV

C. 仅 II、III

D. 仅 II、IV

2014真题

7. 对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的序列是_____。

A. 95, 22, 91, 24, 94, 71

B. 92, 20, 91, 34, 88, 35

C. 21, 89, 77, 29, 36, 38

D. 12, 25, 71, 68, 33, 34

2011真题

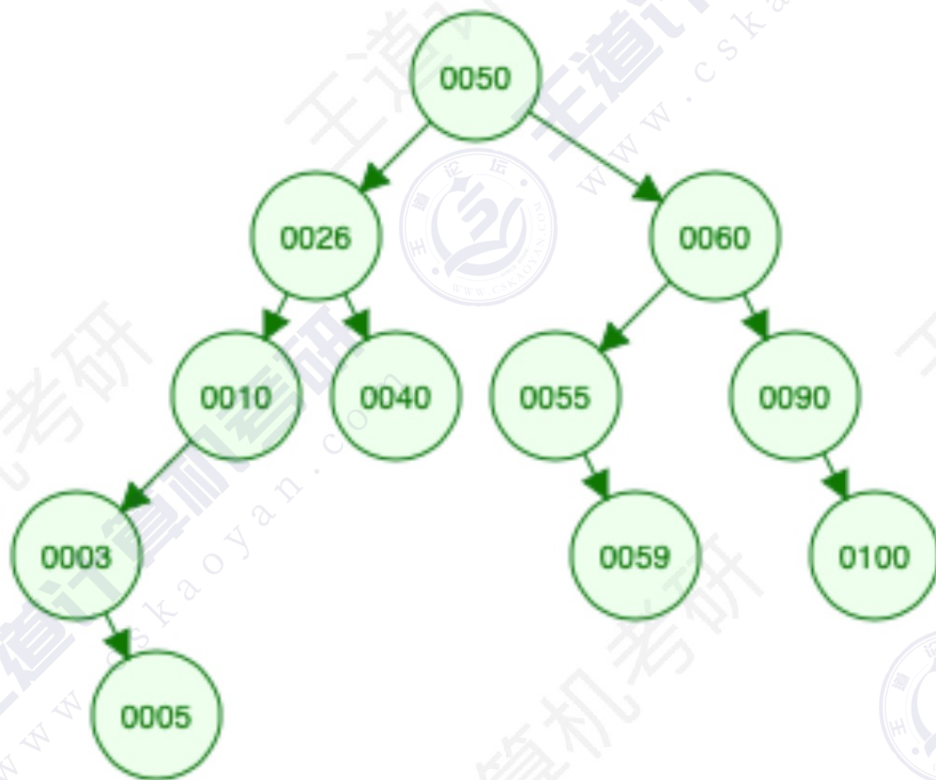
大题的出题点

2019真题

42. (10 分) 请设计一个队列, 要求满足: ①初始时队列为空; ②入队时, 允许增加队列占用空间; ③出队后, 出队元素所占用的空间可重复使用, 即整个队列所占用的空间只增不减; ④入队操作和出队操作的时间复杂度始终保持为 $O(1)$ 。请回答下列问题:
- (1) 该队列应该选择链式存储结构, 还是顺序存储结构?
 - (2) 画出队列的初始状态, 并给出判断队空和队满的条件。
 - (3) 画出第一个元素入队后的队列状态。
 - (4) 给出入队操作和出队操作的基本过程。

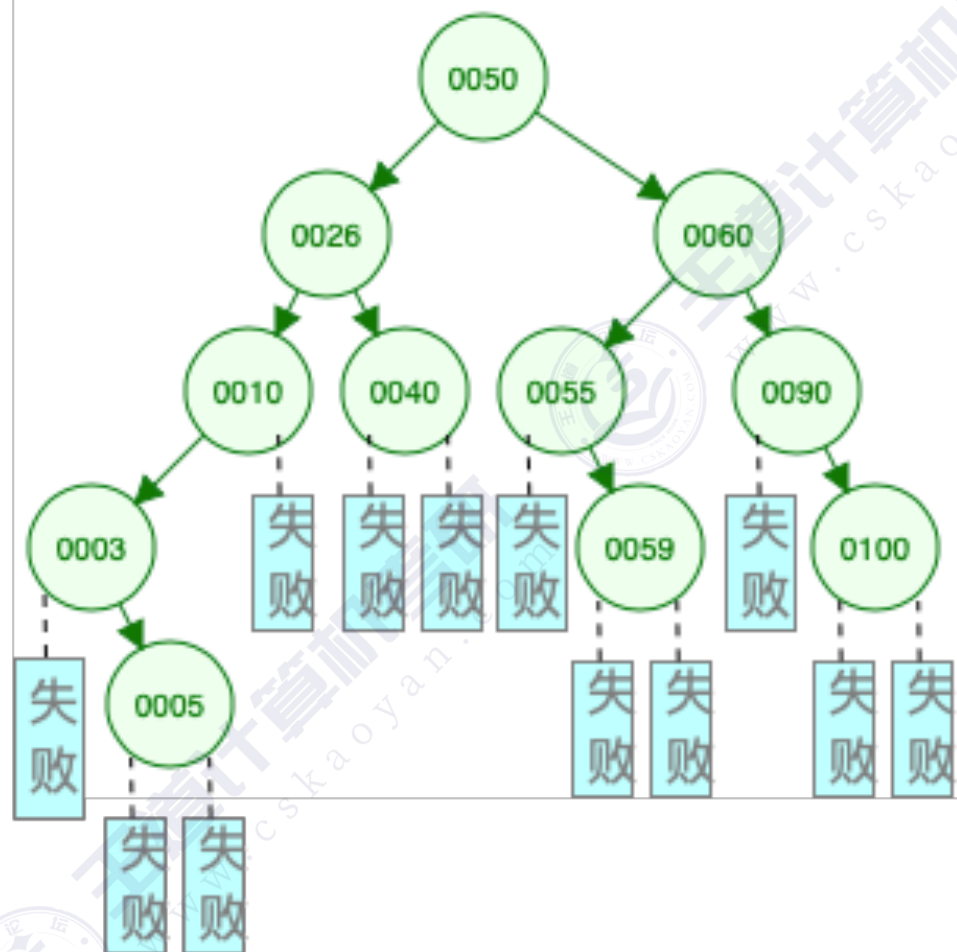
大题的可能出题方式

例：从一棵初始为空的BST开始，依次插入：50、26、10、3、5、60、90、40、55、100、59，最终结果如下



出题方式1：给你一个插入序列，
让你画出二叉排序树

大题的可能出题方式



出题方式2：基于你构造的二叉排序树，
计算查找成功、查找失败的ASL

$$ASL_{成功} = \frac{1}{11} (1 \times 1 + 2 \times 2 + 3 \times 4 + 4 \times 3 + 5 \times 1) = \frac{34}{11}$$

$$ASL_{失败} = \frac{1}{12} (3 \times 5 + 4 \times 5 + 5 \times 2) = \frac{45}{12} = 3.75$$

考点 28:平衡二叉树

• 小题4	• 小题4		• 小题4	• 小题3		• 小题4		
2009	2010	2011	2012	2013	2014	2015	2016	2017
	• 小题4		• 小题6					
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: 小题×7

平衡二叉树

【考点笔记】平衡二叉树的定义

定义结点左子树与右子树的高度差为该结点的平衡因子。

平衡二叉树或者是一棵空树，或者是具有下列性质的二叉树：它的左子树和右子树都是平衡二叉树，且左子树和右子树的高度差的绝对值不超过 1。

注意：我们通常将平衡二叉树默认为二叉排序树，故平衡二叉树的概念包含两个方面平衡、有序。

平衡二叉树

【考点笔记】平衡二叉树的插入与旋转

最小不平衡子树：在插入路径上离插入结点最近的平衡因子的绝对值大于1的结点作为根的子树。如图 4-5 虚线框内为最小不平衡子树。

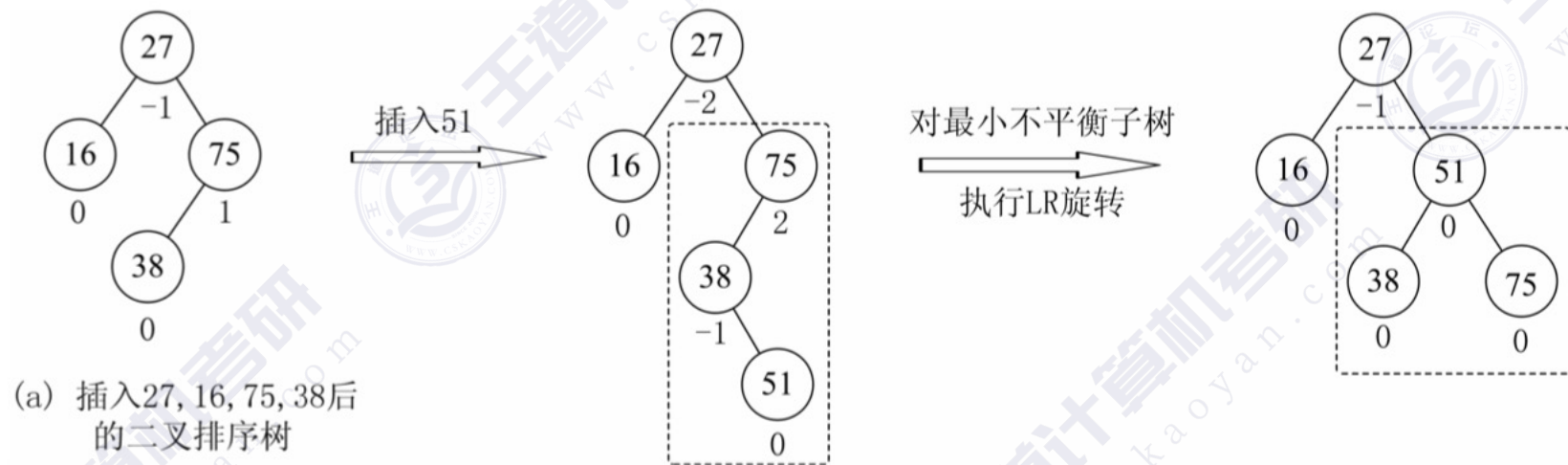


图 4-5 最小不平衡子树

平衡二叉树的插入过程：

- ① 按二叉排序树的插入算法插入一个结点；
- ② 若原平衡二叉树在新结点插入后不再平衡，需要找到最小不平衡子树进行调整。

平衡二叉树

① LL 平衡旋转（右单旋转）。由于在 A 的左孩子(L)的左子树(L)上插入新结点，A 的平衡因子由 1 增至 2，导致以 A 为根的子树失去平衡，需要一次向右的旋转操作。

将 A 的左孩子 B 向右上旋转代替 A 成为根结点，将 A 结点向右下旋转成为 B 的右子树的根结点，而 B 的原右子树则为 A 结点的左子树，如图 4-6 所示。

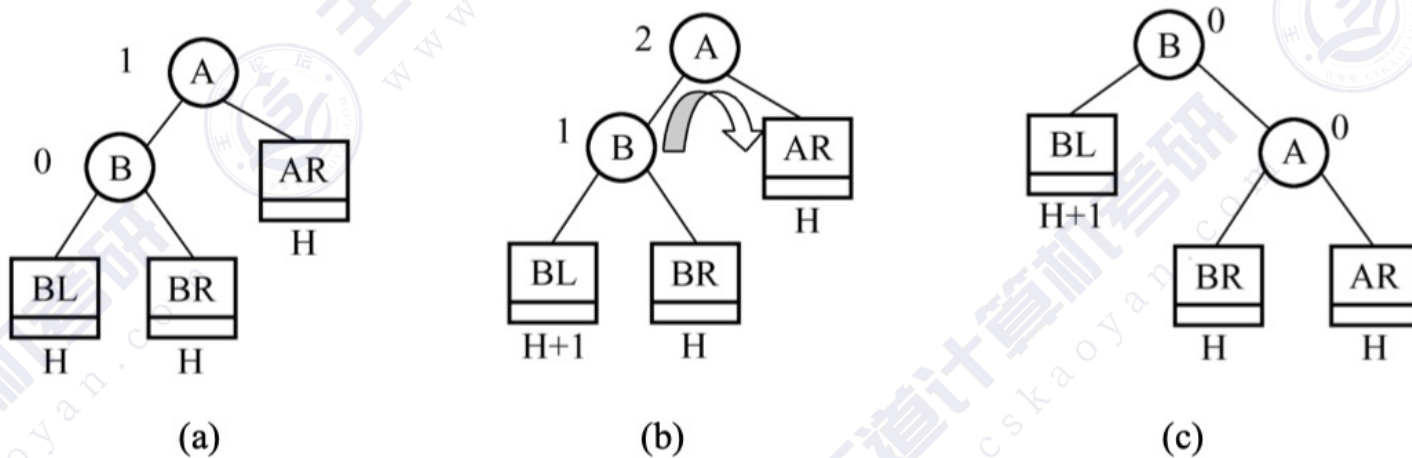
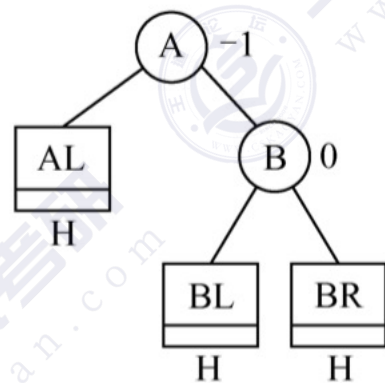


图 4-6 LL 平衡旋转

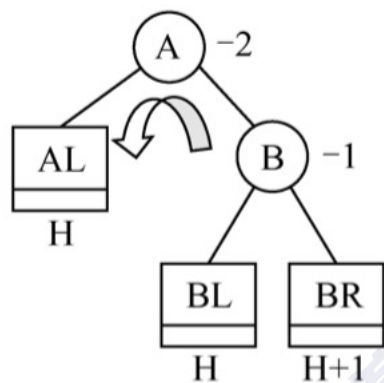
平衡二叉树

② **RR 平衡旋转**（左单旋转）。由于在 A 的右孩子(R)的右子树(R)上插入新结点，A 的平衡因子由-1 减至-2，导致以 A 为根的子树失去平衡，需要一次向左的旋转操作。

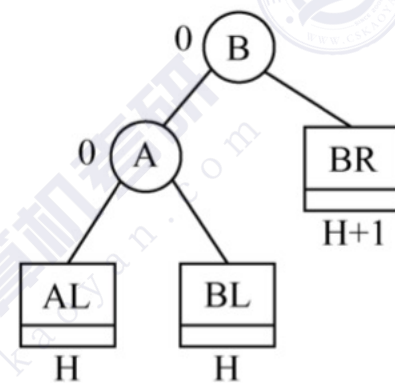
将 A 的右孩子 B 向左上旋转代替 A 成为根结点，将 A 结点向左下旋转成为 B 的左子树的根结点，而 B 的原左子树则作为 A 结点的右子树，如图 4-7 所示。



(a) 插入结点前



(b) 插入结点导致不平衡



(c) RR 旋转（左单旋转）

图 4-7 RR 平衡旋转

平衡二叉树

③ LR 平衡旋转（先左后右双旋转）。由于在 A 的左孩子(L)的右子树(R)上插入新结点，A 的平衡因子由 1 增至 2，导致以 A 为根的子树失去平衡，需要进行两次旋转操作。

先将 A 的左孩子 B 的右子树的根结点 C 向左上旋转提升到 B 结点的位置，然后再把该 C 结点向右上旋转提升到 A 结点的位置，如图 4-8 所示。

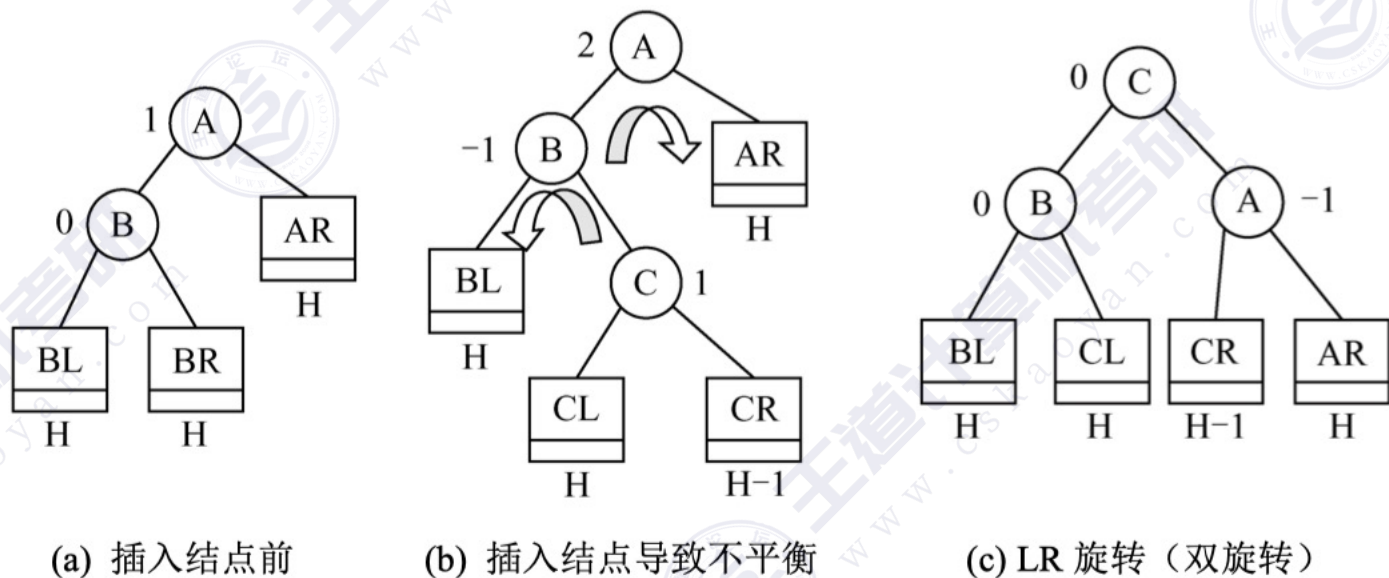


图 4-8 LR 平衡旋转

平衡二叉树

④ RL 平衡旋转（先右后左双旋转）。由于在 A 的右孩子(R)的左子树(L)上插入新结点，A 的平衡因子由-1 减至-2，导致以 A 为根的子树失去平衡，需要进行两次旋转操作。

先将 A 的右孩子 B 的左子树的根结点 C 向右上旋转提升到 B 结点的位置，然后再把该 C 结点向左上旋转提升到 A 结点的位置，如图 4-9 所示。

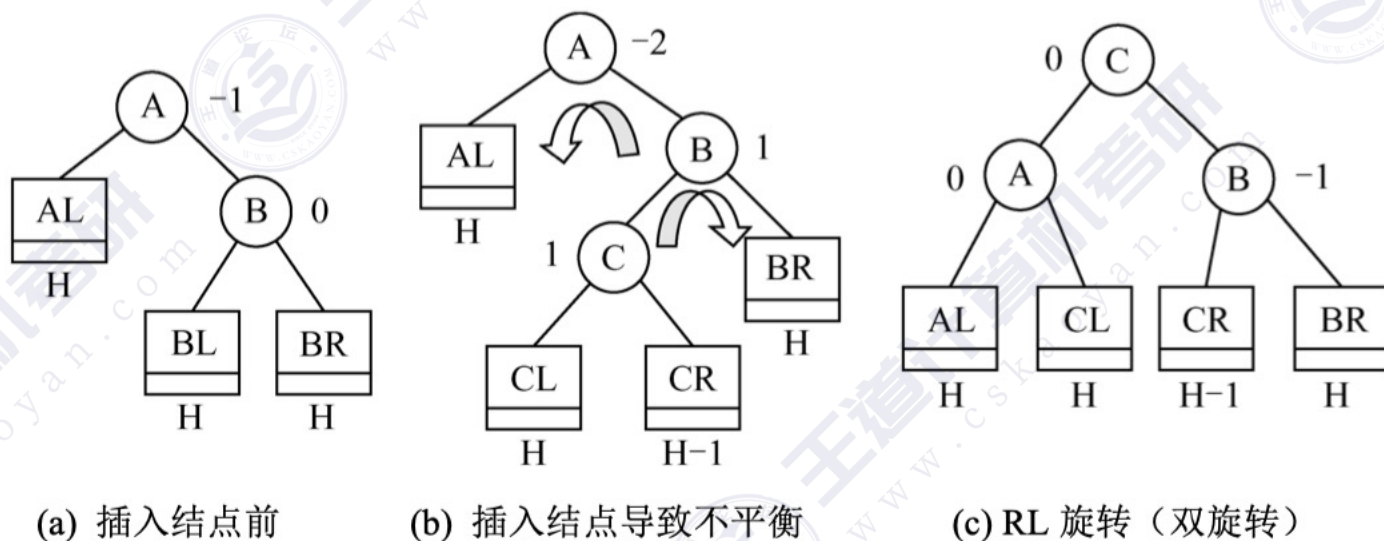


图 4-9 RL 平衡旋转

平衡二叉树

3. 若将关键字 1, 2, 3, 4, 5, 6, 7 依次插入到初始为空的平衡二叉树 T 中, 则 T 中平衡因子为 0 的分支结点的个数是_____。

A. 0

B. 1

C. 2

D. 3

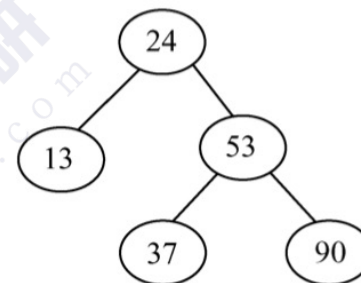
4. 在图 B-1 所示的平衡二叉树中, 插入关键字 48 后得到一棵新平衡二叉树。在新平衡二叉树中, 关键字 37 所在结点的左、右子结点中保存的关键字分别是_____。

A. 13, 48

B. 24, 48

C. 24, 53

D. 24, 90

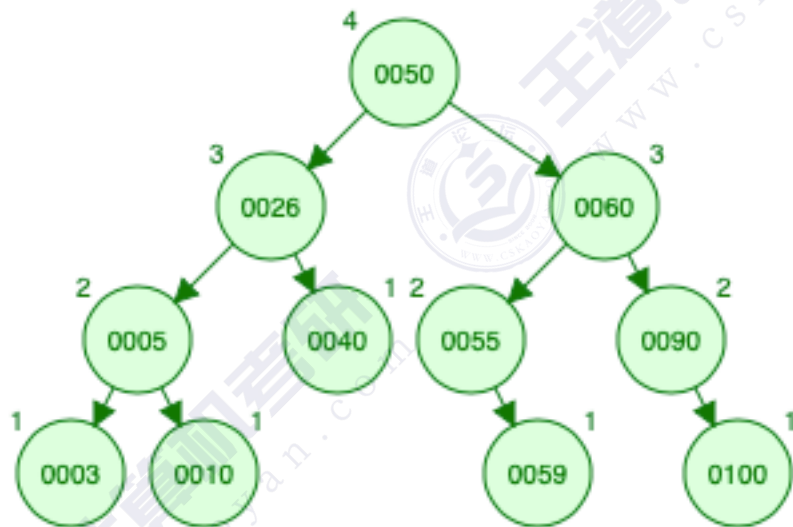


2013真题

2010真题

大题的可能出题方式

例：从一棵初始为空的AVL Trees 开始，依次插入：50、26、10（LL）、3、5（LR）、60、90（RR）、40、55、100、59（RL），画出最终结果如下



出题方式：给你一个插入序列，
让你画出平衡二叉树

上述例子中，插入元素 10、5、90、59 时分别发生了 LL、LR、RR、RL 四种“调整平衡”的情况。
可以通过下面👉这个链接，模拟插入各个元素的过程

<https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>

考点 29:红黑树

2009	2010	2011	2012	2013	2014	2015	2016	2017
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: NULL

红黑树



“红黑树”命题重点——选择题

1.考定义、性质

2.考简单的插入操作

红黑树的定义

红黑树是二叉排序树



左子树结点值 $\leq$ 根结点值 $\leq$ 右子树结点值

注：相应地，包含关键字的结点可称为“内部节点”

与普通BST相比，有什么要求



- ①每个结点或是红色，或是黑色的
- ②根节点是黑色的
- ③叶结点（外部结点、NULL结点、失败结点）均是黑色的
- ④不存在两个相邻的红结点（即红结点的父节点和孩子结点均是黑色）
- ⑤对每个结点，从该节点到任一叶结点的简单路径上，所含黑结点的数目相同

左根右，根叶黑
不红红，黑路同

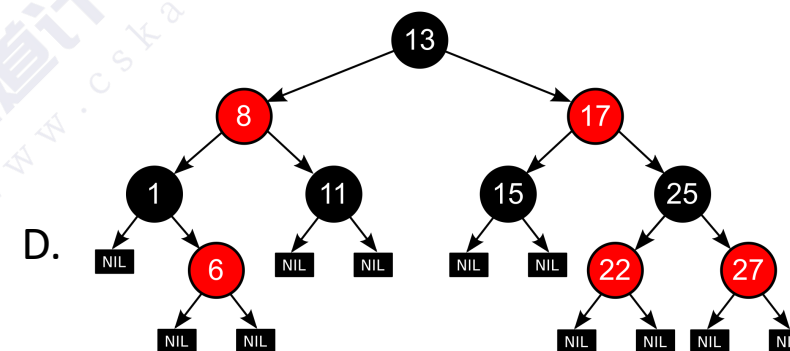
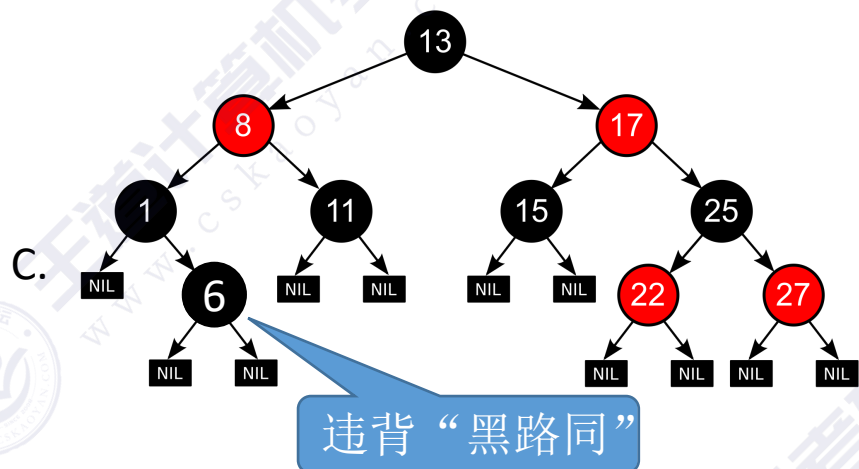
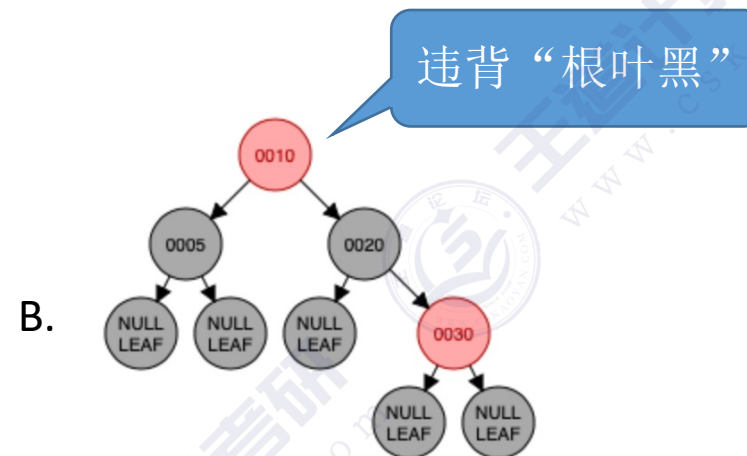
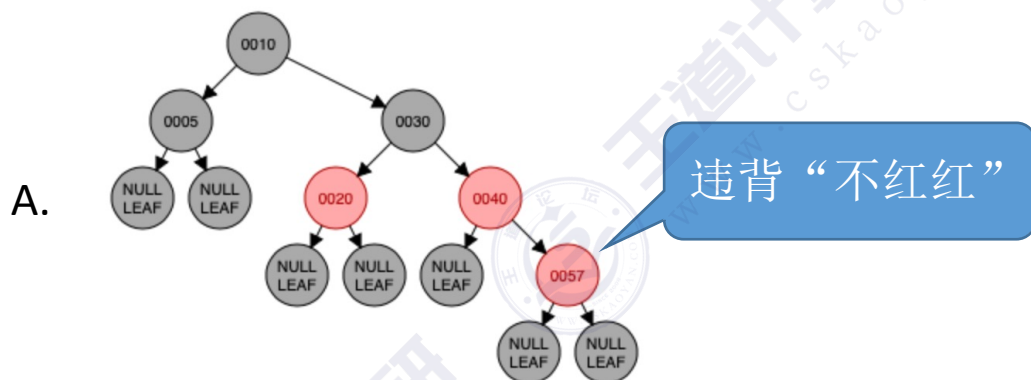


张口就是freestyle

```
struct RBnode {           // 红黑树的结点定义
    int key;               // 关键字的值
    RBnode* parent;        // 父节点指针
    RBnode* lChild;        // 左孩子指针
    RBnode* rChild;        // 右孩子指针
    int color;             // 结点颜色，如：可用 0/1 表示 黑/红，也可使用枚举型enum表示颜色
};
```

红黑树怎么考？——选择题：定义、性质

下面这些选项中，满足“红黑树”定义的是 (D)



红黑树怎么考？——选择题：定义、性质

下列说法中，正确的是？（ ）

下列说法中，错误的是？（ ）

- ✓ • 红黑树是一种特殊的二叉查找树
- ✓ • 红黑树中的某棵子树（如果有的话），也一定是二叉查找树
- ✗ • 红黑树中的某棵子树（如果有的话），也一定是红黑树
- ✗ • 红黑树中的某棵子树（如果有的话），一定不是红黑树
- ✓ • 红黑树中的某棵子树（如果有的话），不一定是红黑树

- ✓ • 非空红黑树的结点要么是红色，要么是黑色
- ✗ • 在一棵红黑树中，如果一个结点是黑的，那么它的孩子结点（如果有的话）一定是红的
- ✓ • 在一棵红黑树中，如果一个结点是红的，那么它的孩子结点（如果有的话）一定是黑的
- ✓ • 红黑树的查找路径上，不允许出现两个连续的红结点
- ✗ • 红黑树中，红结点的数量不会超过内部结点总数的一半
- ✓ • 红黑树的查找路径上，红结点的数量不会超过一半
- ✗ • 红黑树中，任何一个结点的左右子树高度之差不超过1
- ✓ • 红黑树中，任何一个结点的左右子树高度之差不超过2倍
- ✓ • 在一棵红黑树中，如果所有结点都是黑的，那么它的形态一定是满二叉树
- ✗ • 红黑树的任何一个分支结点都有两个非空孩子结点



举个例子？

红黑树怎么考？——选择题：定义、性质

下列说法中，正确的是？（ ）

下列说法中，错误的是？（ ）

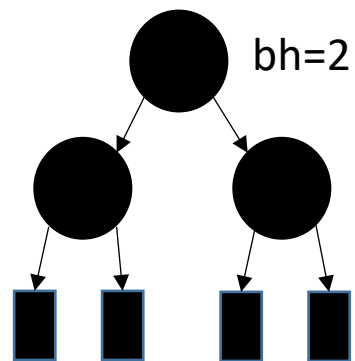
- ✓ 若红黑树根节点黑高度为 h ，则内部结点数（含关键字的结点）最少有 $2^h - 1$ 个
- ✓ 若红黑树根节点黑高度为 h ，则内部结点数（含关键字的结点）最多有 $2^{2h} - 1$ 个
- ✓ 包含 n 个关键字的红黑树，高度不超过 $O(\log_2 n)$
- ✗ 红黑树的查询效率一般要优于含有相同结点数的 AVL 树
- ✓ 红黑树与AVL树的查找、插入、删除操作，最坏时间复杂度数量级相同
- ✓ 红黑树比AVL树更适用于插入、删除操作频繁的应用场景



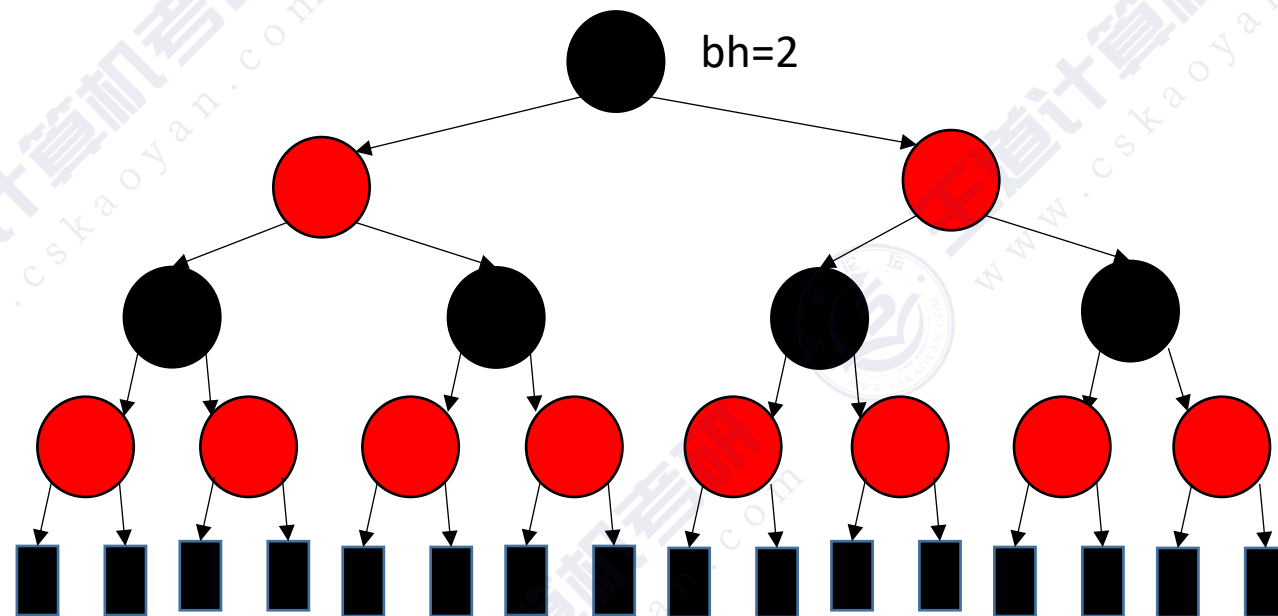
举个例子？

		BST	AVL树	红黑树
最坏时间复杂度	Search (查)	$O(n)$	$O(\log_2 n)$	$O(\log_2 n)$
	Insert (插)	$O(n)$	$O(\log_2 n)$	$O(\log_2 n)$
	Delete (删)	$O(n)$	$O(\log_2 n)$	$O(\log_2 n)$

与“黑高”相关的推论



根节点黑高=2，
内部结点数最
少的情况——全
黑满二叉树



根节点黑高=2，内部结点数最多的情况

结点的**黑高** bh —— 从某结点出发（不含该结点）到达任一叶结点的路径上黑结点总数

若根节点黑高为 h ，内部结点数（关键字）最少有 $2^h - 1$ 个

若根节点黑高为 h ，内部结点数（关键字）最多有 $2^{2h} - 1$ 个

红黑树怎么考？——选择题：简单的插入

- 先查找，确定插入位置（原理同二叉排序树），插入新结点
- 新结点是根——染为黑色
- 新结点非根——染为红色
 - 若插入新结点后依然满足红黑树定义，则插入结束
 - 若插入新结点后不满足红黑树定义，需要调整，使其重新满足红黑树定义
 - 黑叔：旋转+染色
 - LL型：右单旋，父换爷+染色
 - RR型：左单旋，父换爷+染色
 - LR型：左、右双旋，儿换爷+染色
 - RL型：右、左双旋，儿换爷+染色
 - 红叔：染色+变新
 - 叔父爷染色，爷变为新结点

如何调整：看新结点叔叔的脸色



左根右，根叶黑
不红红，黑路同



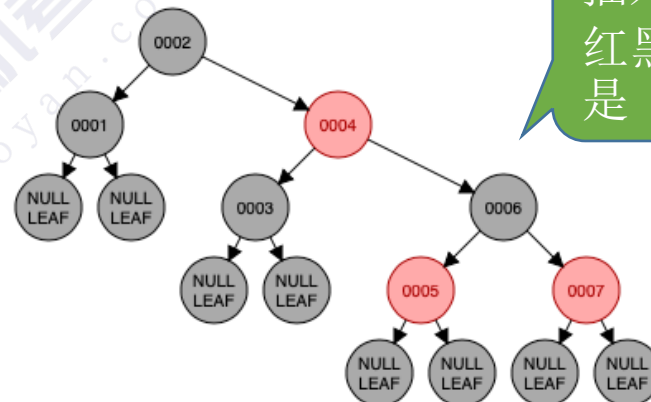
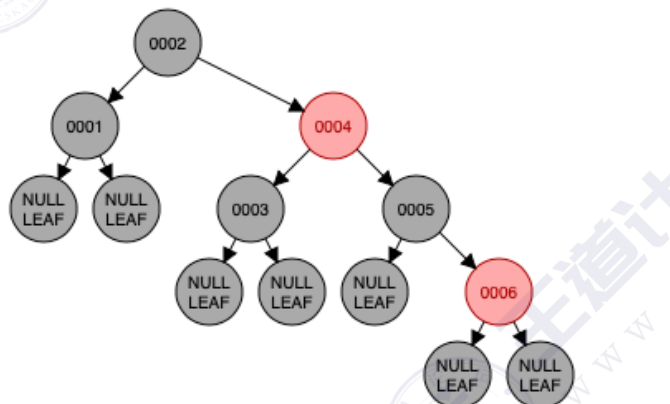
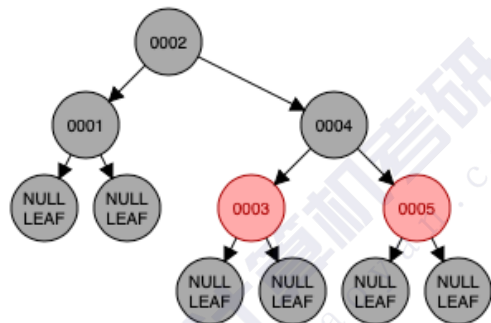
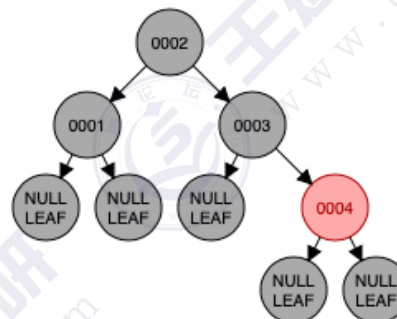
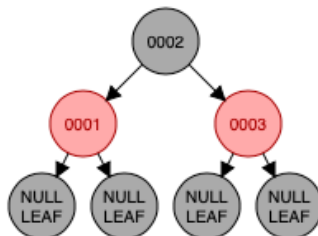
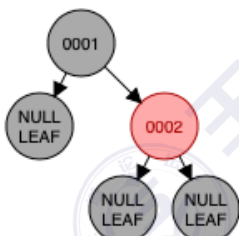
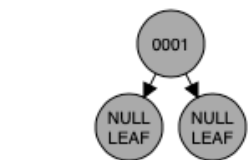
黑叔转，再染色
红叔染，爷变新

张口就是freestyle

红黑树怎么考？——选择题：简单的插入

若将关键字 1, 2, 3, 4, 5, 6, 7 依次插入到初始为空的红黑树中, 插入完成后红色结点个数是 (C)

A. 1 B. 2 C. 3 D. 4



插入关键字 8,
红黑树的样子
是 () ?

补充选择题出题点:

- 在红黑树中, 同样的关键字按照不同的顺序插入, 最终得到的红黑树形态可能会不同
- 在红黑树中, 先删除一个关键字, 紧接着再插入相同的关键字, 红黑树的形态可能会发生改变

红黑树练习方法（插入操作）

课件中的例子，插入：20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 18

Red/Black Tree

①打上勾，显示空叶结点

Insert Delete Find Print ☒ Show Null Leaves

③插入关键字

②选择 pause，使用 Step Forward/Step Back 进行单步演示

Animation Completed

Skip Back Step Back pause Step Forward Skip Forward

Animation Speed w: 800 h: 300 Change Canvas Size Move Controls

Algorithm Visualizations

自我训练方法：你可以自己设计一些插入序列，每一次插入后，都先思考应该如何调整，然后在网站中演示验证。插入新元素时，尽可能覆盖 黑叔LL/RR/LR/RL、红叔 的情况。

<https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>

知识回顾



红黑树

左根右，根叶黑
不红红，黑路同
黑叔转，再染色
红叔染，爷变新



考点 30:B 树及其基本操作、B+树的基本概念

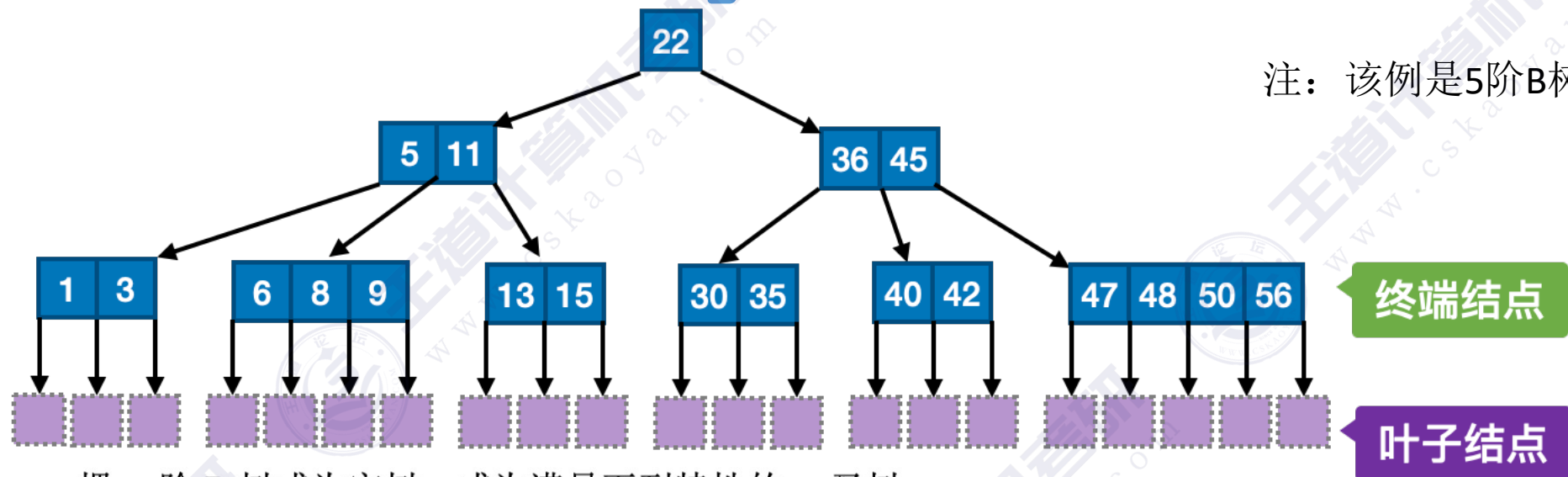


• 小题8			• 小题9	• 小题10	• 小题9		• 小题10	• 小题9
2009	2010	2011	2012	2013	2014	2015	2016	2017
• 小题8		• 小题10	• 小题9	• 小题8	• 小题7		• 小题8	
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频： 小题×12

B 树及其基本操作、B+树的基本概

念



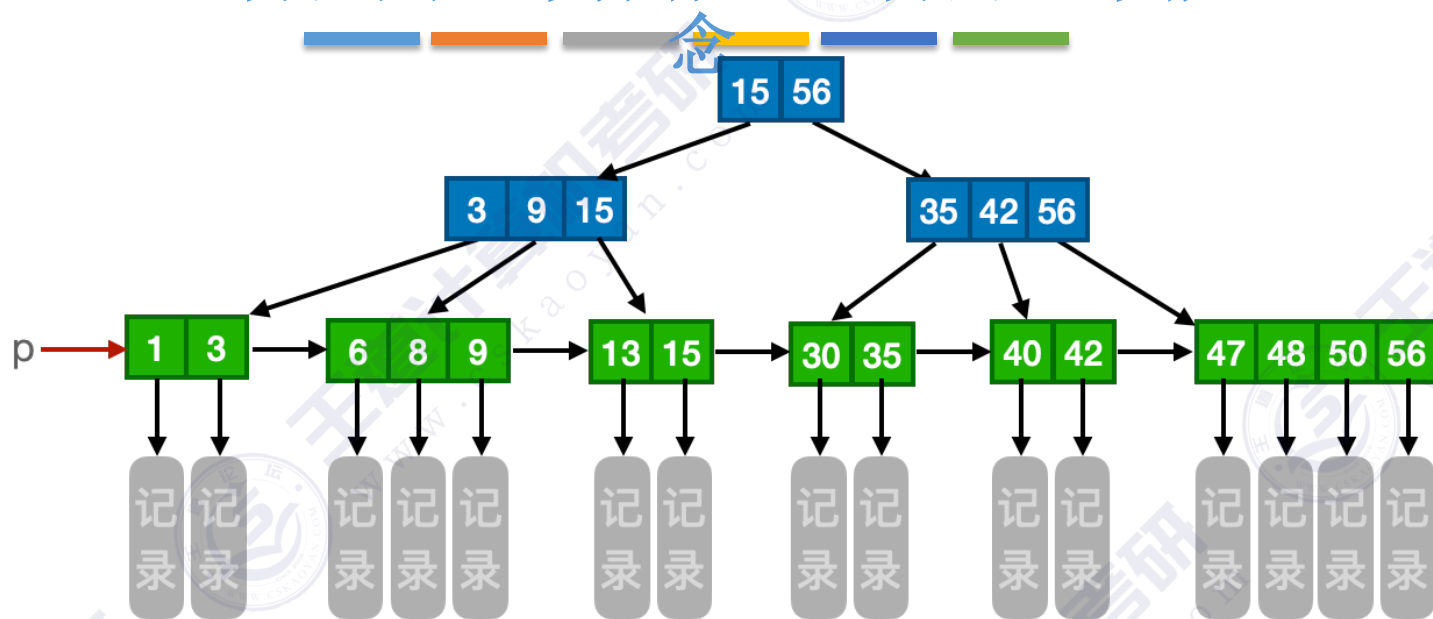
一棵 m 阶 B 树或为空树，或为满足下列特性的 m 叉树：

- ① 若根结点不是终端结点，则至少有两棵子树，至多有 m 棵子树。
- ② 除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树，至多有 m 棵子树。
- ③ 所有的叶结点都出现在同一层次上，并且不带信息（可看成失败结点）。
- ④ 所有非叶结点的结构如下：

n	P_0	K_1	P_1	K_2	P_2	$\dots$	K_n	P_n
-----	-------	-------	-------	-------	-------	---------	-------	-------

- a) P_{i-1} 所指子树中所有结点的关键字均小于 K_i
- b) P_i 所指子树中所有结点的关键字均大于 K_i

B 树及其基本操作、B+树的基本概



注：以上是一棵4阶B+树

一棵 m 阶的 B+树需满足下列条件：

- ① 每个分支结点最多有 m 棵子树。
- ② 非叶根结点至少有两棵子树，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树。
- ③ 结点的子树个数与关键字个数相等。
- ④ 所有叶结点包含全部关键字及指向相应记录的指针，而且叶结点中将关键字按大小顺序排列，并且相邻叶结点按大小顺序相互链接起来。
- ⑤ 所有分支结点（可看成是索引的索引）中仅包含它的各个子结点（即下一级的索引块）中关键字的最大值及指向其子结点的指针。

B 树及其基本操作、B+树的基本概

念

【考点笔记】B 树与 B+树的区别

- ① 在 B+树中，具有 n 个关键字的结点只含有 n 棵子树，即每个关键字对应一棵子树；而在 B 树中，具有 n 个关键字的结点含有 $(n+1)$ 棵子树。
- ② 在 B+树中，非根结点关键字个数 n 的范围是 $\lceil m/2 \rceil \leq n \leq m$ （根结点： $1 \leq n \leq m$ ），在 B 树中，非根结点关键字个数 n 的范围是 $\lceil m/2 \rceil - 1 \leq n \leq m - 1$ （根结点： $1 \leq n \leq m - 1$ ）。
- ③ 在 B+树中，所有非叶结点仅起到索引作用，即结点中的每个索引项只含有对应子树的最大关键字和指向该子树的指针，不含有该关键字对应记录的存储地址。
- ④ 在 B+树中，叶结点包含了全部关键字，即其他非叶结点中的关键字包含在叶结点中；而在 B 树中，叶结点包含的关键字和其他结点包含的关键字是不重复的。

B 树及其基本操作、B+树的基本概念

8. 高度为 5 的 3 阶 B 树含有的关键字个数至少是_____。

A. 15

B. 31

C. 62

D. 242

2018真题

10. B+树不同于 B 树的特点之一是_____。

A. 能支持顺序查找

B. 结点中含有关键字

C. 根结点至少有两个分支

D. 所有叶结点都在同一层上

2016真题

8. 下列叙述中，不符合 m 阶 B 树定义要求的是_____。

A. 根结点最多有 m 棵子树

B. 所有叶结点都在同一层上

C. 各结点内关键字均升序或降序排列

D. 叶结点之间通过指针链接

2009真题

B 树及其基本操作、B+树的基本概

【考点笔记】B-树的查找

B-树的查找算法：

- ① 在 B-树中找结点（磁盘上进行），当查找到叶结点时，查找失败。
- ② 在结点内的多关键字有序表中查找关键字（内存中进行）。
 - a) 先在有序表中进行查找，若找到则查找成功；
 - b) 否则，根据找到的指针信息到所指的子树，执行①。

对于含有 n 个关键字的 B-树的查找，磁盘 I/O 次数也就是树的高度 h （不包含叶结点）满足： $h \leq \log_{\lceil m/2 \rceil}((n+1)/2) + 1$ 。

B 树及其基本操作、B+树的基本概

【考点笔记】B-树的插入

B 树的插入过程如下：

- 1) 查找：利用 B-树查找算法，找出插入该关键字的最底层中某个非叶结点。
- 2) 插入：当插入后的结点关键字个数小于 m ，则可以直接插入；如果等于 m ，则必须对结点进行分裂。

分裂的过程及方法如图 6-1 所示（3 阶 B 树）：

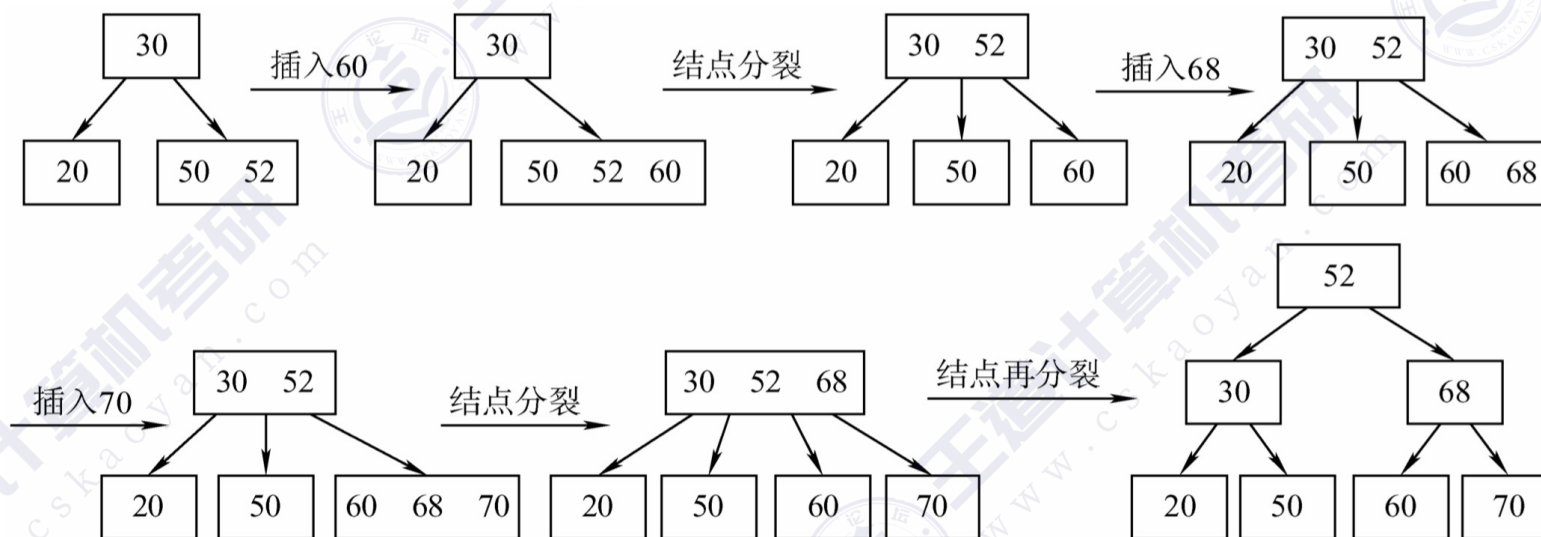


图 6-1 结点分裂示意图

若此时导致其父结点的关键字个数也超过了上限，则继续进行这种分裂操作
若最终使得根结点分裂，B 树的高度增 1。

B 树及其基本操作、B+树的基本概

念

【考点笔记】B-树的删除

为使删除后的结点中的关键字个数 $\geq \lceil m/2 \rceil - 1$ ，将涉及结点的“合并”问题。

(1) 当被删除的关键字 k 不在非叶终端结点中时：

① 如果小于（大于） k 的子树中关键字个数 $> \lceil m/2 \rceil - 1$ ，则找出 k 的前驱（后继）值 k' ，并且用 k' 来取代 k ，再递归地按此方法删除 k' 。

② 如果前后两个子树中关键字个数均为 $\lceil m/2 \rceil - 1$ ，则将两个子结点合并，直接删除 k 。如图 6-2 所示为某 4 阶 B 树的一部分。

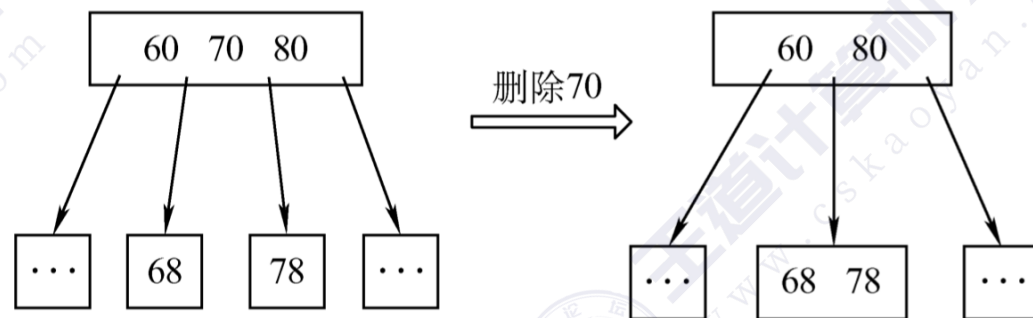


图 6-2 B-树中删除非终端结点关键字的合并

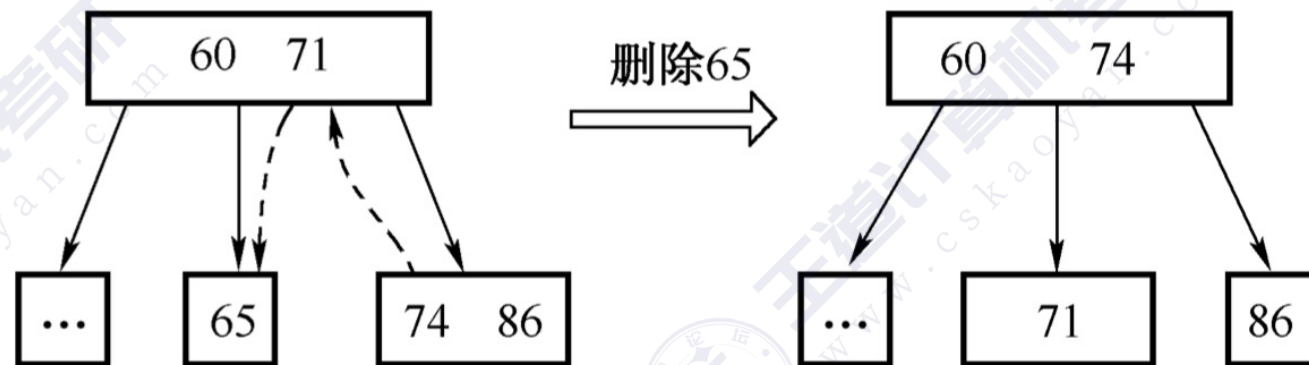
B 树及其基本操作、B+树的基本概

念

(2) 当被删除的关键字 k 在非叶终端结点中时:

① 若该结点的关键字个数 $> \lceil m/2 \rceil - 1$, 直接删去该关键字。

② 若该结点的关键字个数 $= \lceil m/2 \rceil - 1$, 且与此结点相邻的右 (左) 兄弟结点的关键字个数 $\geq \lceil m/2 \rceil$, 需要调整该结点、右 (左) 兄弟结点以及其双亲结点 (父子换位法), 以达到新的平衡, 如图 6-3(a)所示。



(a) 兄弟够借

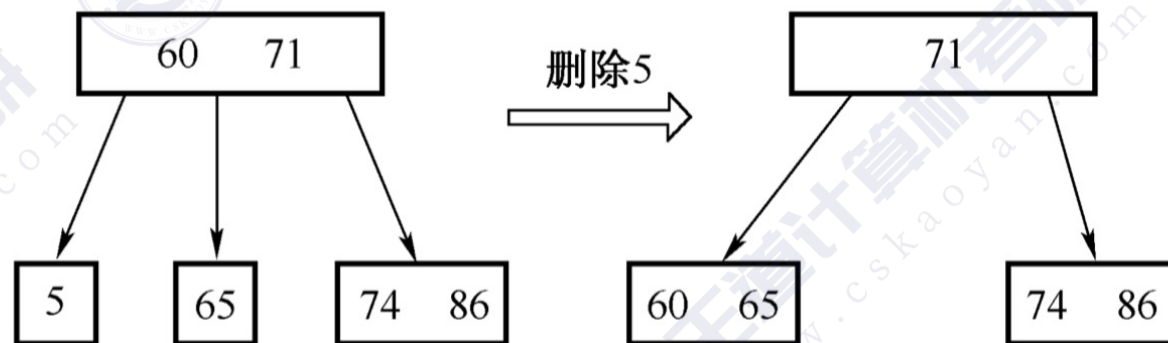
B 树及其基本操作、B+树的基本概念

念

③ 若该结点关键字个数= $\lceil m/2 \rceil - 1$ ，且与该结点相邻的右（左）兄弟结点的关键字个数= $\lceil m/2 \rceil - 1$ ，则将关键字删除后与右（左）兄弟结点及双亲结点中的关键字进行合并，如图 6-3(b)所示。

在合并的过程中，若此时导致其父结点的关键字个数也不满足 B-树的定义，则继续进行这种合并操作。

若最终使得根结点被合并，B 树高度减 1。



(b) 兄弟不够借

图 6-3 B 树中删除终端结点关键字的示意图

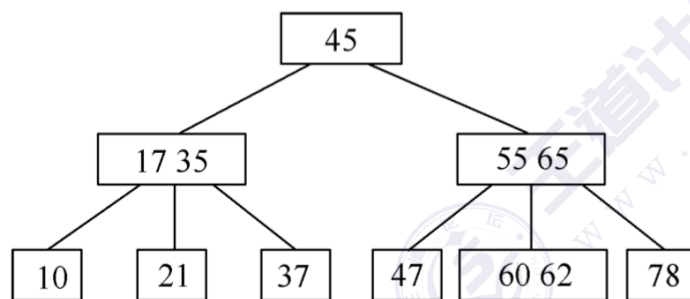
B 树及其基本操作、B+树的基本概念

10. 依次将关键字 5,6,9,13,8,2,12,15 插入初始为空的 4 阶 B 树后, 根结点中包含的关键字是

- A. 8
- B. 6,9
- C. 8,13
- D. 9,12

2020真题

9. 已知一棵 3 阶 B-树, 如下图所示。删除关键字 78 得到一棵新 B-树, 其最右叶结点中的关键字是_____。



- A. 60
- B. 60, 62
- C. 62, 65
- D. 65

2012真题

考点 31:散列(Hash)表

	•应用题41	•小题9			•小题8			
2009	2010	2011	2012	2013	2014	2015	2016	2017
•小题9	•小题8			•小题9	•小题9	•应用题42	•小题9	
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: 小题×7、应用题×2

散列(Hash)表

【考点笔记】相关基本概念

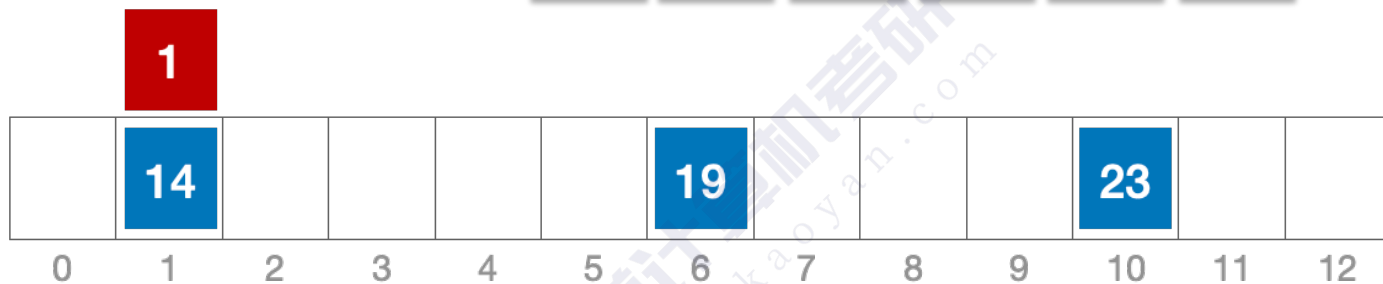
术语	定义及说明
散列表	散列表建立了关键字和存储地址之间的一种直接映射关系,把关键字映射成其对应地址的函数称为散列函数,记为 Hash(key)。
冲突	散列函数把两个或两个以上的不同关键字映射到同一地址。
冲突处理方法	开放定址法:容易产生聚集现象。拉链法:不会产生聚集现象。
聚集(堆积)	非同义词之间争夺一个地址的情况。

【考点笔记】散列函数的构造方法

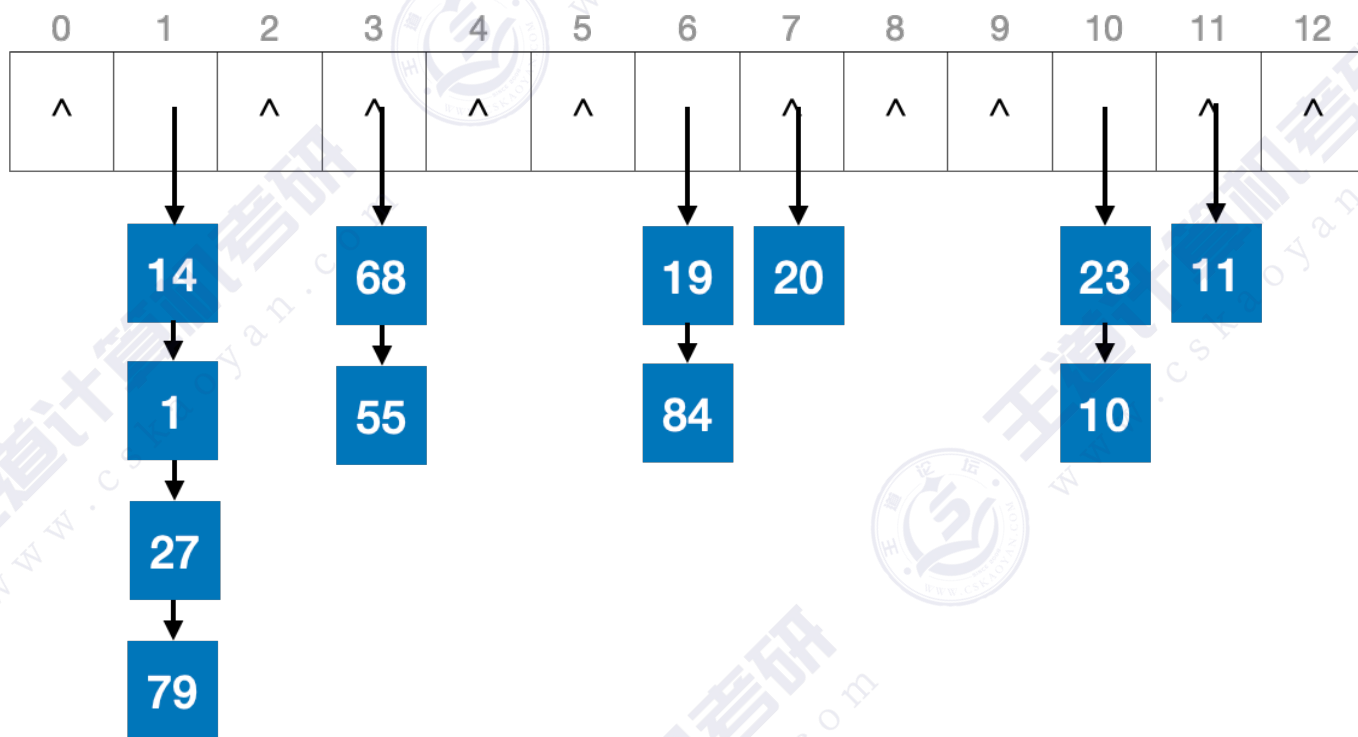
最常用的是除留余数法,假设散列表表长为 m , 取一个不大于 m 但最接近或等于 m 的质数 p , 利用以下公式把关键字转换成散列地址。散列函数为

$$H(\text{key}) = \text{key} \% p$$

散列(Hash)表



$19\%13=6$
 $14\%13=1$
 $23\%13=10$
 $1\%13=1$



$19\%13=6$ $27\%13=1$
 $14\%13=1$ $55\%13=3$
 $23\%13=10$ $11\%13=11$
 $1\%13=1$ $10\%13=10$
 $68\%13=3$ $79\%13=1$
 $20\%13=7$
 $84\%13=6$

散列(Hash)表

【考点笔记】处理冲突的方法

处理冲突方法	说明
开放定址法	发生冲突后第 i 次探测的散列地址为 $H_i = (H(\text{key}) + d_i) \% m$ 式中, $i=1, 2, \dots, k$ ($k \leq m-1$); m 为散列表表长; d_i 为增量序列。 ● 当 $d_i=1, 2, \dots, m-1$, 称为线性探测法。冲突发生时, 顺序查看表中下一个单元, 直到找出一个空闲单元或查遍全表。线性探测法易造成“聚集”现象。 ● 当 $d_i=1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$, 其中 $k \leq m/2$, m 必须是一个可以表示成 $4k+3$ 的质数, 称二次探测法
拉链法	把所有散列地址相同的记录存储在同一个线性链表中, 这个线性链表由其散列地址唯一标识。拉链法适用于经常进行插入和删除的情况

散列(Hash)表

【考点笔记】平均查找长度

平均查找长度是查找过程中进行关键字的比较次数的平均值。其数学定义为

$$ASL = \sum_{i=1}^n P_i C_i$$

式中， P_i 是查找第*i*个数据元素的概率， C_i 是找到第*i*个数据元素所需的比较次数。

散列(Hash)表

8. 现有长度为 11 且初始为空的散列表 HT, 散列函数是 $H(key) = key \% 7$, 采用线性探查(线性探测再散列)法解决冲突。将关键字序列 87, 40, 30, 6, 11, 22, 98, 20 依次插入到 HT 后, HT 查找失败的平均查找长度是

- A. 4 B. 5.25 C. 6 D. 6.29

2019真题

9. 现有长度为 7、初始为空的散列表 HT, 散列函数 $H(k) = k \% 7$, 用线性探测再散列法解决冲突。将关键字 22, 43, 15 依次插入到 HT 后, 查找成功的平均查找长度是_____。

- A. 1.5 B. 1.6 C. 2 D. 3

2018真题

	22	43	15			
0	1	2	3	4	5	6

ASL成功 $= (1+2+3)/3=2$

ASL失败 $= (1+4+3+2+1+1+1)/7=1.85$

重要概念: 装填因子 $= 3/7=0.42$